DBT ON SNOWFLAKE: DEVELOPMENT LIFECYCLE WITH AZURE DEVOPS

Prepared for Estia Health

This page shows how Estia Health can run dbt changes from a branch to production using Azure Repos and Azure Pipelines. Engineers work on a branch, build it into their own schemas, open a pull request and merge. Two pipelines then release `main` to DEV and, after approval, to PROD, and a third rebuilds both from new data on a schedule. Snowflake doesn't need tasks or a Git integration. DEV and PROD are separate databases in one Snowflake account, kept apart by separate service users and roles. Object, user and branch names match the walkthrough and can be changed to suit your naming standards.



One Snowflake account · ESTIA_DBT_DEMO_DEV: per-engineer and CI schemas, shared SILVER/GOLD · ESTIA_DBT_DEMO_PROD: SILVER/GOLD · separated by service users and roles

The process, step by step

1. **Engineer pushes a branch** named `feature/jsmith/falls-count` to Azure Repos.
2. **Pipeline 1 (dbt-ci) runs automatically.** It uploads the branch from the agent with `snow dbt deploy` as `DBT_JSMITH` and builds it into `JSMITH_SILVER/_GOLD`. The engineer reads results in the pipeline log.
3. **Pull request.** The branch policy runs Pipeline 1 again into `CI_PR_<id>_SILVER/_GOLD`. If any dbt test fails, the PR can't be completed. A reviewer must approve.
4. **Merge to main** triggers **Pipeline 2 (dbt-release)**. Its DEV stage deploys and builds shared DEV.
5. **Approval** on the `estia-prod` environment, then the PROD stage deploys **the same commit** and builds PROD.
6. **Data refresh**: **Pipeline 3 (dbt-refresh)** runs `dbt build` in DEV and PROD on a schedule (e.g. after the Data Factory load), using the version last released. It deploys nothing and needs no approval.

Runs as: `SVC_ADO_DBT_DEV` (DEV only) for Pipeline 1 and the DEV stages; `SVC_ADO_DBT_PROD` for the PROD stages. No passwords: both sign in with workload identity.

To set up once	
Azure	A Managed Identity or App Registration in Microsoft Entra for each service connection.
Azure DevOps	Service connections `snowflake-dev` and `snowflake-prod` (workload identity federation); environments `estia-dev` and `estia-prod` with approvers; the Snowflake CLI extension; branch policy on `main` with Pipeline 1 as build validation and a required reviewer; branch control on `snowflake-prod`.
Snowflake	Service users `SVC_ADO_DBT_DEV` and `SVC_ADO_DBT_PROD`, each trusting one service connection (OIDC issuer and subject copied from Azure DevOps); roles `ESTIA_DBT_DEV_DEPLOY` and `ESTIA_DBT_PROD_DEPLOY`, and `ESTIA_DBT_ENGINEER` to read the results (`setup/02_roles_and_service_users.sql`); optionally a warehouse per environment. No tasks, Git repository objects or tokens.
dbt project	Models, tests, `dbt_projects_profiles.yml` and `env.yml` with `personal`, `dev` and `prod` targets.

Why two service users in one account: Pipeline 1 runs on any branch. Its user can only reach DEV, so unreviewed code can never be deployed to PROD. PROD is reachable only through `snowflake-prod`, which is locked to `main` and sits behind the approval.

THE AZURE DEVOPS PIPELINES

Pipeline 1: dbt-ci (build and test branches and pull requests)

Purpose	Prove a change builds and passes every dbt test before it can reach `main`.
Triggers	Push to `feature/*`; **build validation** on PRs into `main` (branch policy); **Run pipeline** on any branch with a name.
Signs in as	Service connection `snowflake-dev` → `SVC_ADO_DBT_DEV`, role `ESTIA_DBT_DEV_DEPLOY`. Can't see PROD.
Builds into	Push: `<NAME>_SILVER/_GOLD`, name taken from `feature/<name>/…`. PR: `CI_PR_<id>_SILVER/_GOLD`.
Outcome	Green: PR can be completed once approved. Red: PR blocked; the log shows which model or test failed.
Housekeeping	Drop `DBT_CI_PR_<id>` and its schemas after the PR closes (a cleanup step or a weekly job).

```
trigger: { branches: { include: [ feature/* ] } }
steps:
  - task: ConfigureSnowflakeCLI@1        # OIDC via service connection
    inputs: { useWorkloadIdentity: true, connectedServiceName: snowflake-dev }
  - script: |
      NAME=...   # CI_PR_$(System.PullRequest.PullRequestId) or name from branch
      snow dbt deploy DBT_$NAME --source . -x \
        --database ESTIA_DBT_DEMO_DEV --schema DBT --default-target dev
      snow dbt execute -x --database ESTIA_DBT_DEMO_DEV --schema DBT \
        --env-vars "{\"DBT_DEV_PREFIX\": \"$NAME\"}" DBT_$NAME build --target personal
    env: { SNOWFLAKE_TOKEN: $(SNOWFLAKE_TOKEN) }
```

Pipeline 2: dbt-release (release main to DEV, then PROD)

Purpose	Release each merge to shared DEV, then to PROD after approval.
Trigger	Merge (push) to `main` only.
Stage DEV	Environment `estia-dev`, connection `snowflake-dev`. Deploy `ESTIA_MEDALLION` to `ESTIA_DBT_DEMO_DEV.DBT`, build `--target dev`.
Stage PROD	Runs only if DEV succeeded. Environment `estia-prod` with **required approvers**. Connection `snowflake-prod` → `SVC_ADO_DBT_PROD`, limited to `main` by branch control. Deploy to `ESTIA_DBT_DEMO_PROD.DBT`, build `--target prod`.
Outcome	One run per merge, the same commit in both databases, and a deployment history per environment in Azure DevOps.

```
trigger: { branches: { include: [ main ] } }
stages:
  - stage: DEV
    jobs: [ { deployment: dev, environment: estia-dev, ... } ]
      # ConfigureSnowflakeCLI@1 with snowflake-dev, then:
      # snow dbt deploy ESTIA_MEDALLION --database ESTIA_DBT_DEMO_DEV ...
      # snow dbt execute -x ... ESTIA_MEDALLION build --target dev
  - stage: PROD
    dependsOn: DEV
    jobs: [ { deployment: prod, environment: estia-prod, ... } ]  # approval
      # ConfigureSnowflakeCLI@1 with snowflake-prod, then:
      # snow dbt deploy ESTIA_MEDALLION --database ESTIA_DBT_DEMO_PROD ...
      # snow dbt execute -x ... ESTIA_MEDALLION build --target prod
```

Pipeline 3: dbt-refresh. A daily schedule on `main` (`always: true`), or Run pipeline. Its DEV and PROD stages run `snow dbt execute -x ... ESTIA_MEDALLION build` with no deploy and no approval, so new bronze data reaches silver and gold with the code already released. A failed test fails the run and leaves gold as it was. Data Factory can start it at the end of its load instead of the schedule. **Why a third pipeline:** releases change code and need a PROD approval each time; refreshes change only data, run daily and must not wait for an approver or redeploy. Keeping them apart gives each pipeline one trigger and one purpose. The snippets above are outlines; the complete pipelines are in `pipelines/`. Snowflake recommends deploying dbt projects to production with the Snowflake CLI from a CI/CD pipeline ("Deploy dbt project objects", Snowflake documentation).

© 2026 Snowflake Inc. All Rights Reserved.